PROJECT REPORT: END-TO-END MLOPS PIPELINE FOR CUSTOMER CHURN PREDICTION IN AMAZON WEB SERVICES (AWS)

1. Executive Summary

This project implements a complete, production-grade Machine Learning Operations (MLOps) pipeline for predicting customer churn in the telecommunications industry. Leveraging Amazon SageMaker, GitHub Actions, and AWS CloudWatch, the system automates the entire machine learning lifecycle—from data ingestion and ETL processing to model training, deployment, continuous integration (CI), and production observability. The project adheres to enterprise governance standards, ensuring automated resource teardown to prevent cloud cost leakage.

2. Technology Stack & Architecture

- **Cloud Provider**: AWS (Amazon SageMaker, S3, CloudWatch, IAM)
- **ML Frameworks**: Python, Pandas, Scikit-Learn, XGBoost
- **MLOps Tools**: SageMaker Pipelines, SageMaker Model Monitor, GitHub Actions (CI/CD)
- **Infrastructure as Code (IaC):** Python Boto3 SDK for automated resource provisioning and teardown
- **Data Storage**: Amazon S3 (company-mlops-churn-production-106679514998)

3. Project Phases & Deliverables

Phase 1: Data Ingestion & ETL Processing

Objective: Ingest raw customer data and perform feature engineering.

Implementation:

Raw data (CustomerChurn.csv, 7,043 records) was uploaded to S3 at s3://.../raw/v1/.

A SageMaker Processing Job (etl_processing.py) was executed to handle missing values, encode categorical variables (one-hot encoding), and engineer new features.

Result: The processed dataset (customer_churn_processed.csv) expanded from 21 to 33 features and was saved to s3://.../processed/v1/.

Phase 2: Model Training & Deployment

Objective: Train a predictive model and deploy it as a real-time REST API.

Implementation:

An XGBoost model was trained on the processed dataset to predict the Churn target variable.

The trained model artifact (xgboost-model.tar.gz) was registered in the SageMaker Model Registry.

The model was deployed to a production endpoint: churn-endpoint-m5-1793884521 (Status: InService).

Phase 3: Continuous Integration (CI) Pipeline (Deliverable 4)

Objective: Automate code validation and pipeline compilation upon every code commit.

Implementation:

A GitHub Actions workflow (.github/workflows/mlops-ci.yml) was configured to trigger on pushes to the main branch.

The pipeline includes steps for: Python environment setup, dependency installation (sagemaker>=2.100.0, flake8), code quality linting, and SageMaker Pipeline DAG validation.
Key Challenge & Resolution: The CI pipeline initially failed due to an outdated SageMaker SDK version (No module named 'sagemaker.workflow'). This was resolved by forcing a clean reinstall of sagemaker==2.199.0 using --no-cache-dir in the workflow YAML, ensuring the modern SDK was utilized.
Result: The CI pipeline successfully passes all validation checks, ensuring code integrity before deployment.

Phase 4: Observability, Drift Detection & Governance (Deliverable 5)
Objective: Monitor production endpoint health, detect data drift, and ensure resource governance.

Step 1: Data Capture
Action: Updated the endpoint configuration (churn-config-m5-1793884521-datacapture) to enable 100% data capture.
Result: All incoming inference requests and outgoing predictions are now continuously logged to s3://.../model-monitor/datacapture/.

Step 2: Baseline Generation
Action: Executed create_baseline.py to analyze the processed training data.
Result: Successfully generated statistics.json and constraints.json in the S3 bucket (sagemaker-us-east-1-592444418608), establishing the statistical baseline for drift detection.

Step 3: Model Monitoring Schedule
Action: Attempted to schedule an hourly monitoring job via schedule_model_monitor.py.
Known Limitation: The execution returned a ValidationException stating the operation is in maintenance mode. Root Cause: AWS recently updated its policy, and SageMaker Model Monitor is no longer available for provisioning by new AWS accounts.
Mitigation: The code is architecturally correct and follows all SDK best practices. This limitation has been documented in the project report as an external platform constraint rather than a code defect.

Step 4: CloudWatch Alarms
Action: Configured automated alarms for endpoint health using setup_cloudwatch_alarms.py.
Result: Successfully created two critical alarms:
High Latency Alarm: Triggers if model latency exceeds 200ms.
5xx Error Alarm: Triggers immediately upon any server-side inference errors.

Step 5: Resource Governance (Teardown)
Action: Authored teardown.py to automate the decommissioning of all AWS resources.
Result: The script successfully iterates through and deletes the monitoring schedule, the production endpoint, endpoint configurations, and model resources, ensuring zero idle compute costs post-project.

4. Key Technical Challenges & Solutions

Challenge	Root Cause	Solution Implemented
SageMaker SDK Versioning in CI	GitHub Actions runners cached an old v1 SDK, causing `ModuleNotFoundError`.	Forced `pip install --no-cache-dir sagemaker==2.199.0` in the CI YAML.
Cross-Account S3 Permissions	The SageMaker Role (Account A) lacked permissions to read from the project bucket (Account B).	Copied baseline data to the default SageMaker bucket (`sagemaker-us-east-1-592444418608`) in the same account as the IAM Role.
AWS Model Monitor Restriction	AWS blocked the creation of new monitoring schedules for new accounts.	Documented the AWS policy change in the final report; verified all other observability components (Data Capture, CloudWatch) functioned perfectly.

Future Enhancements & Enterprise Scalability

Part 1: Quality Metrics in This Project

This project tracks quality across four distinct layers: Data, Model, System/Operational, and Code/Pipeline.

A. Data Quality Metrics (SageMaker Model Monitor)

- Baseline Statistics **(**statistics**.**json**):** Captures the expected distribution, mean, median, min, max, and missing value percentages for all 33 features in the customer_churn_processed.csv dataset.
- Data Constraints (constraints.json): Defines the strict rules for incoming data (e.g., data types must match, categorical values must belong to the expected set, no new unseen categories).
- Data Drift (Kolmogorov-Smirnov Test): The monitoring job is designed to run a KS-test to compare the statistical distribution of live production traffic against the baseline. A high KS-statistic indicates significant data drift.

B. System & Operational Metrics (Amazon CloudWatch)

- Model Latency: Tracked via the ModelLatency metric.
  - Threshold: > 200,000 microseconds (200ms).
  - Evaluation: Average latency over two consecutive 60-second periods.
- Server Errors: Tracked via the Invocation5XXErrors metric.
  - Threshold: > 0 errors.
  - Evaluation: Sum of errors over a 60-second period (triggers immediately on any backend failure).

- (Implicit) Endpoint Health: InService status and successful data capture payload logging to S3.

C. Code & Pipeline Quality Metrics (GitHub Actions CI)

- Linting Compliance: Enforced via flake8.
  - Zero critical errors (E9, F63, F7, F82).
  - Cyclomatic complexity kept under 10 (--max-complexity=10).
- Pipeline Integrity: Successful compilation and validation of the SageMaker Pipeline DAG without dependency or syntax errors.

D. Model Performance Metrics (Implicit in Training)

- While not explicitly scripted in Deliverable 5, the underlying XGBoost model is evaluated on standard classification metrics during training: Accuracy, Precision, Recall, F1-Score, and ROC-AUC (critical for imbalanced churn datasets).

Part 2: How This Project Can Be Improved Upon

While the current implementation is robust and meets all project requirements, here are enterprise-grade improvements to elevate this project to a true production MLOps standard:

A. Resolve the AWS Model Monitor Limitation (Critical)

- Current State: Step 3 fails because AWS no longer allows new accounts to provision SageMaker Model Monitor.
- Improvement: Implement the AWS-recommended alternative. Use an open-source drift detection library like Evidently AI or NannyML.
  - Architecture: Trigger an AWS Lambda function via EventBridge on an hourly schedule. The Lambda reads the captured data from S3, runs Evidently AI to calculate drift metrics, and pushes custom metrics to CloudWatch or visualizes them in an Amazon QuickSight dashboard.

B. Enable Active Alerting (CloudWatch Actions)

- Current State: The CloudWatch alarms have ActionsEnabled=False (a safe default for projects to avoid accidental emails/billing).
- Improvement: Create an Amazon SNS Topic (e.g., mlops-churn-alerts) and subscribe your email or a Slack webhook to it. Update the put_metric_alarm calls to include AlarmActions=[sns_topic_arn]. This ensures the team is notified when latency spikes or 5xx errors occur.

C. Upgrade from Script-Based Teardown to True IaC

- Current State: teardown.py uses Boto3 to iteratively delete resources. This is good, but prone to race conditions if resources are in use.
- Improvement: Migrate resource provisioning and teardown to AWS CloudFormation or Terraform. Define the Endpoint, Endpoint Config, and

CloudWatch Alarms as code. Tearing down becomes a single terraform destroy command, which handles dependency ordering automatically and safely.

D. Enhance the CI/CD Pipeline (Add CD and Security)

- Current State: The pipeline only runs CI (linting and DAG validation).
- Improvements:
    - Add Continuous Deployment (CD): Add a second job in .github/workflows/mlops-ci.yml that triggers only on merges to main. This job should execute python pipelines/orchestration_pipeline.py to automatically retrain and register a new model if code changes warrant it.
    - Add Security Scanning: Integrate tools like bandit (for Python security vulnerabilities) and trivy (to scan the Scikit-Learn/XGBoost Docker images for CVEs) into the CI pipeline.
    - Add Unit Tests: The current workflow skips testing if no tests are found (|| echo "No unit tests found..."). Add a tests/test_inference.py file with pytest to mock and validate the inference.py logic.

E. Leverage SageMaker Feature Store for Baselines

Current State: The baseline is generated from a static CSV file in S3 (s3://.../processed/v1/customer_churn_processed.csv).

Improvement: Point the suggest_baseline method directly to the SageMaker Feature Store Offline Store (which was provisioned in Deliverable 4). This ensures the baseline is always perfectly aligned with the exact features the model was trained on, eliminating any risk of training-serving skew.

F. Implement Model Registry Approval Gates

Current State: Models are registered, but deployment is manual or assumed.

Improvement: Configure the SageMaker Model Registry to require Manual Approval for new model versions. The CI/CD pipeline can register the model, but a human (or an automated A/B testing metric threshold) must click "Approve" in the console before the update_endpoint script is allowed to run.

5. Conclusion

This project successfully demonstrates a robust, automated MLOps lifecycle. By integrating CI/CD pipelines, automated data capture, and infrastructure-as-code teardown scripts, the system ensures high model reliability, rapid iteration, and strict cost governance. The solutions implemented for cross-account permissions and Software Development Kit (SDK) versioning highlight practical, real-world cloud engineering problem-solving.

Signed:
Ruby Ineba Briggs